

A STORYBOARD IN 9 FRAMES

# Prompting is not the skill you think it is

*Asked nicely.  
Ignored politely.*

*ALWAYS.  
NEVER.  
YOU MUST.*

MORE INPUT

≠



REJECT

A CHECK OUTSIDE THE PROMPT



A prompt shapes what the model is likely to produce. Shaping isn't enforcing, so anything that must hold needs a check that lives outside the prompt.

# Five things a prompt can say.

1

## Intent

*what you're trying to achieve (the part skipped most)*

⋮

2

## Constraints

*what must not happen*

⋮

3

## Examples

*two or three showing the pattern*

⋮

4

## Requested shape

*the format you want back*

⋮

5

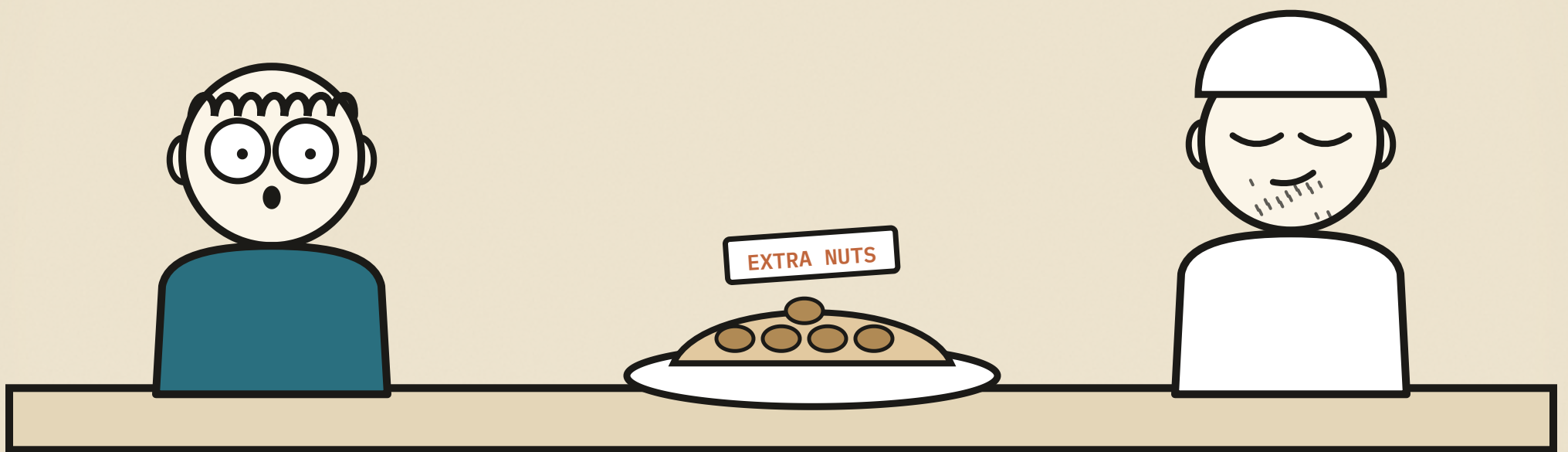
## Priorities

*what wins when instructions pull apart*

Each of these narrows the field of likely answers. None of them closes it.

# Asked. Cooked. Nobody tasted.

NO NUTS!!!



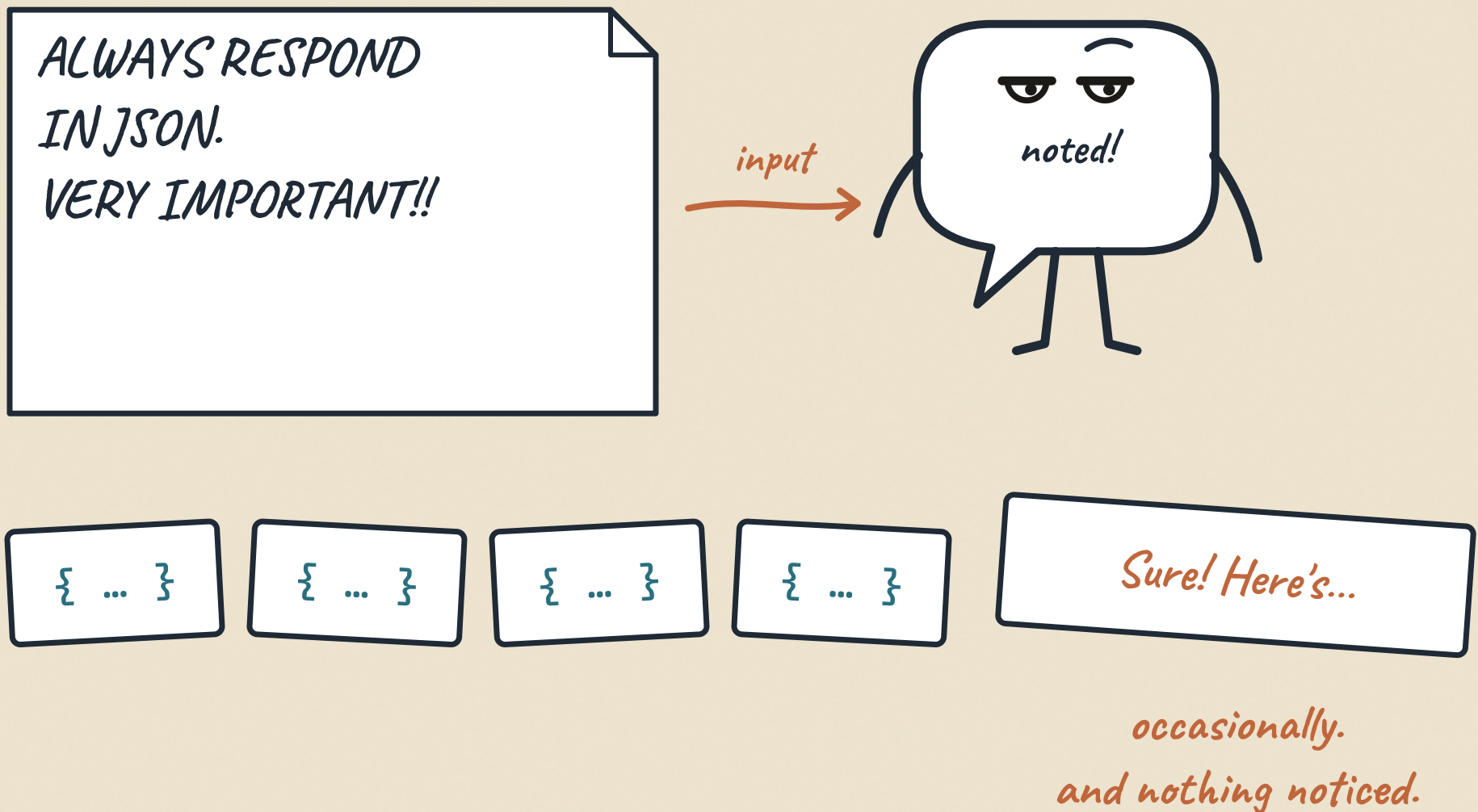
*(the taster is on holiday)*

---

*The order said NO NUTS. Nobody checked.*

Generation is conditioned by the instruction, and no separate step afterwards verifies that the output satisfied it.

# Shouting isn't checking.



~~“ALWAYS”~~ → **a validator**

“Always” and “never” are more input, and more input is not a check. If you want a step that notices and objects, you have to build it.

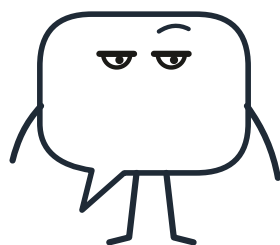
# A wish list isn't a contract.



*Dear model, please be JSON. Love, me.*

A prompt tells the model what you want. A contract gives software something it can check, and reject.

# Ask, or check?



## A prompt

*tells the model  
what you want*

- ☒ order\_id: string
- ☒ amount: number
- ☐ reason: string



## A contract

*gives software  
something to check*

Plain language can be the human-facing interface. Something checkable still has to sit underneath it.

# Read the gap, then pick a fix.

*asked: 3 slogans*



*got: 7, and a poem*

*the skill is reading this gap*

## **Wrong tone, length**

*say the thing  
you didn't say*

## **Wrong shape**

*now and then? use  
a schema + validator*

## **Wrong facts**

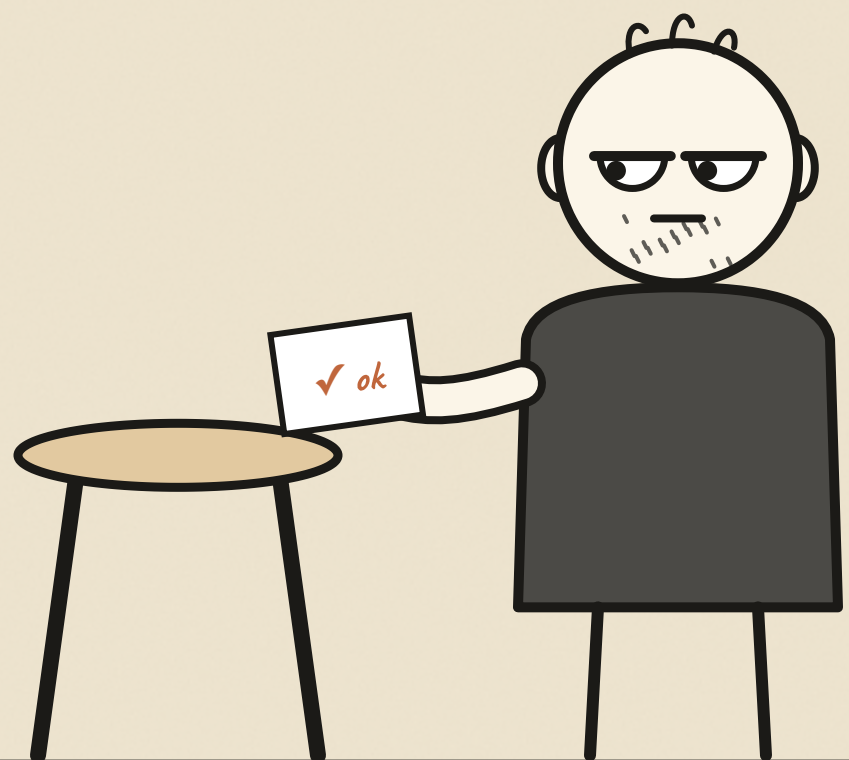
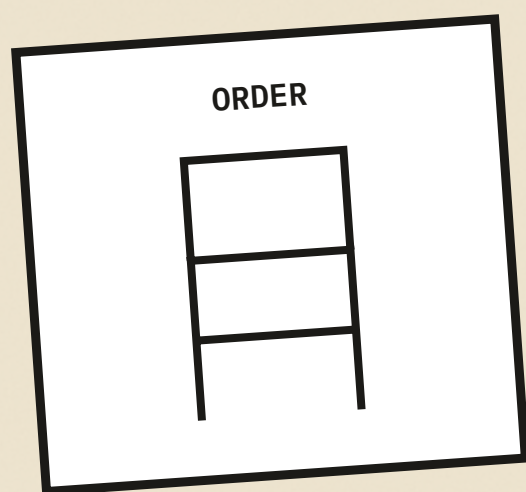
*trace the evidence  
before the wording*

## **Mostly right**

*wrong sometimes?  
design for that*

Right most of the time and wrong occasionally, in ways that matter:  
that's when to stop editing the prompt.

# The skill is noticing.



*Ordered a chair. Got a stool. Signed for it.*

The real skill is reading the gap between what you asked for and what came back, then deciding what should enforce the difference.

# Treat a prompt like code.

prompt.md

v7: shorter

v6: tone fix

v5: examples

IN GIT

- ☒ refund request
- ☒ angry customer
- ☒ empty input
- ☐ very long thread

*rerun the same set  
after every edit*

If a prompt does real work, handle it like code: keep it in version control, note what a change should do, and rerun a small fixed set of inputs after every edit.

## Read the whole explanation

[pruningmypothos.com/systems/  
prompting-is-not-the-skill-you-think-it-is/](https://pruningmypothos.com/systems/prompting-is-not-the-skill-you-think-it-is/)

